Oracle SQL: Tables, DML, Transactions, User Access and Views

Study Notes (Part 3) with Easy Definitions

1. Foreign Key and Table Creation Practice

1.1 Foreign Key

A **foreign key** creates a link between two tables. The table that holds the foreign key is the **child table**, and the table it refers to is the **parent table**.

> **In simple words:** A foreign key value must already exist in the parent table. This stops a child record from pointing to a record that does not exist.

- The value passed in a foreign key column must exist in the parent column.
- The parent column is normally a primary key (or unique) column.

```
CREATE TABLE dotnetdev
(
 project_id   INT PRIMARY KEY,
 project_name VARCHAR2(50) UNIQUE NOT NULL,
 uselanguage  VARCHAR2(30),
 dev_id       INT,
 CONSTRAINT dev_id_fk FOREIGN KEY (dev_id) REFERENCES developer (dev_id)
);

INSERT INTO dotnetdev VALUES (1000, 'CRM', 'MVC', 101);
INSERT INTO dotnetdev VALUES (1001, 'CRP', 'MVC', 102);   -- works only if dev_id 102 exists in developer

SELECT * FROM dotnetdev;

SELECT * FROM developer JOIN dotnetdev USING (dev_id);
```

Inserting a value that does not exist in the parent table gives an error:

```
INSERT INTO dotnetdev VALUES (1002, 'CRR', 'MVC', 103);
-- Invalid: integrity constraint (SYSTEM.DEV_ID_FK) violated - parent key not found
```

1.2 Understanding Data Type Sizes

Data type	Meaning
NUMBER(5)	A whole number with up to 5 digits.
NUMBER(8,2)	A number with a total of 8 digits, of which 2 are after the decimal point (for example, 123456.78).
VARCHAR2(20)	Text of up to 20 characters.

Data type	Meaning
CHAR(1)	Text of exactly 1 character.
DATE	A date value.

1.3 Practice Tables with Constraints

Each table below uses different constraints. Giving a name to a constraint (with the CONSTRAINT keyword) makes error messages easier to understand.

```
-- 1. Primary key and NOT NULL
CREATE TABLE employeeone (
 employee_id NUMBER(5) CONSTRAINT employee_id_pk PRIMARY KEY,
 first_name  VARCHAR2(20) CONSTRAINT first_name_not_null NOT NULL,
 last_name   VARCHAR2(20) CONSTRAINT last_name_not_null NOT NULL,
 salary      NUMBER(8,2),
 hire_date   DATE
);

DESC employeeone;

-- 2. Department table
CREATE TABLE deptdemo (
 department_id   NUMBER(4) CONSTRAINT deptdemo_id_pk PRIMARY KEY,
 department_name VARCHAR2(30) CONSTRAINT deptdemo_name_not_null NOT NULL,
 manager_id      NUMBER(5),
 location_id     NUMBER(4)
);

-- 3. Primary key, NOT NULL and UNIQUE
CREATE TABLE student (
 student_id     NUMBER(5) CONSTRAINT student_id_pk PRIMARY KEY,
 student_name   VARCHAR2(30) CONSTRAINT student_name_not_null NOT NULL,
 email          VARCHAR2(50) CONSTRAINT email_unique UNIQUE,
 admission_date DATE
);

-- 4. CHECK constraint on price
CREATE TABLE product (
 product_id   NUMBER(5) CONSTRAINT product_id_pk PRIMARY KEY,
 product_name VARCHAR2(50),
 price        NUMBER(8,2) CONSTRAINT price_check CHECK (price > 0),
 quantity     NUMBER(5)
);
```

```
-- 5. UNIQUE mobile number
CREATE TABLE customertbl (
 customer_id   NUMBER(5) CONSTRAINT customer_id_pk PRIMARY KEY,
 customer_name VARCHAR2(50) CONSTRAINT customer_name_not_null NOT NULL,
 mobile_no     NUMBER(10) CONSTRAINT mobile_no_unique UNIQUE
);

-- 6. CHECK constraint on order amount
CREATE TABLE orders (
 order_id     NUMBER(6) CONSTRAINT order_id_pk PRIMARY KEY,
 order_date   DATE,
 customer_id  NUMBER(5),
 order_amount NUMBER(10,2) CONSTRAINT order_amount_check CHECK (order_amount
> 0)
);

-- 7. PRIMARY KEY, UNIQUE, CHECK and DEFAULT together
CREATE TABLE employee_details_demo (
 employee_id NUMBER(5)
  CONSTRAINT employee_id_pk_demo PRIMARY KEY,
 email VARCHAR2(50)
  CONSTRAINT email_uniqueone_demo UNIQUE,
 salary NUMBER(8,2)
  CONSTRAINT salary_amountone_demo CHECK (salary > 5000),
 joining_date DATE DEFAULT CURRENT_DATE
);
```

Constraints without names

Constraints can also be written without a name. In that case, the Oracle server generates a name in the `SYS_Cn` format.

```
CREATE TABLE attendance2 (
 attendance_id   NUMBER(4) PRIMARY KEY,
 student_id      NUMBER(4) UNIQUE,
 attendance_date DATE DEFAULT CURRENT_DATE,
 status          CHAR(1) CHECK (status IN ('P', 'A')),
 marks           NUMBER(3) CHECK (marks BETWEEN 0 AND 100)
);
```

- `status` accepts only `P` (present) or `A` (absent).
- `marks` accepts only values from 0 to 100.

2. Creating a Table Using a Subquery

A new table can be created and filled with data in one step by combining the `CREATE TABLE` statement with the `AS` subquery option.

> **In simple words:** It is a quick way to make a copy of a table (or of selected columns) together with its data.

- The number of specified columns must match the number of columns in the subquery.
- Column names and default values can be defined for the new table.

> **Note:** Only the `NOT NULL` constraints are copied to the new table. Other constraints, such as a primary key or foreign key, are not copied.

Copying the whole department table

```
CREATE TABLE deptcopy
AS
SELECT * FROM departments;

-- Inserting data into the deptcopy table
INSERT INTO deptcopy VALUES (280, 'HR', NULL, 700);
```

Copying only two columns, with new column names

```
CREATE TABLE dcopy (d_id, d_name)
AS
SELECT department_id, department_name FROM departments;

SELECT * FROM dcopy;
```

3. ALTER TABLE, RENAME, TRUNCATE and DROP

3.1 ALTER TABLE Command

The `ALTER TABLE` command is used when we want to make changes in the **structure** of a table.

> **In simple words:** CREATE TABLE builds the table. ALTER TABLE changes the table after it has been built.

ALTER TABLE is used to

- Add a new column.
- Modify an existing column definition.
- Define a default value for the new column.
- Drop a column.
- Rename a column.

- Change a table to read-only status.

A demo table is used for the examples below.

```
CREATE TABLE septaltdemo
(
 a_id   INT,
 a_name VARCHAR2(10)
);

DESC septaltdemo;
```

ADD: adds a new column

```
ALTER TABLE septaltdemo
ADD a_date DATE;

DESC septaltdemo;
```

MODIFY: changes a column definition

MODIFY can change the data type, set a default value or apply constraints.

```
ALTER TABLE septaltdemo
MODIFY a_date VARCHAR2(30);

DESC septaltdemo;
```

> **Note:** The data type of a column can usually be changed only when the column is empty (contains no data).

DROP COLUMN: removes a column

```
ALTER TABLE septaltdemo
DROP COLUMN a_date;

DESC septaltdemo;
```

RENAME COLUMN: changes a column name

```
ALTER TABLE septaltdemo
RENAME COLUMN a_date TO alter_date;
```

SET UNUSED: hides a column

SET UNUSED is used when we do not know whether a column will be needed in the long run. Rather than dropping the column, we can hide it.

```
ALTER TABLE septaltdemo
SET UNUSED (alter_date);
```

Option	Result
DROP	The column is permanently removed.

Option	Result
SET UNUSED	The column is permanently hidden. Once a column is set unused, it cannot be used again.

READ ONLY: protects a table from changes

A read-only table cannot be changed. The values cannot be modified and new values cannot be added, which means both DDL and DML operations are not allowed. To make the table changeable again, set it to READ WRITE.

```
ALTER TABLE septaltdemo READ ONLY;

-- Not allowed while the table is read only
ALTER TABLE septaltdemo RENAME COLUMN alter_date TO alter_date2;

ALTER TABLE septaltdemo READ WRITE;
```

3.2 RENAME Table

The RENAME command changes the name of a table.

```
RENAME septaltdemo TO s_altdemo;
```

3.3 TRUNCATE

TRUNCATE deletes **all rows** from a table while the table structure remains as it is.

> **In simple words:** TRUNCATE empties the table but keeps the empty table. It is like pressing Shift + Delete on the values of a table: they are gone permanently.

- It removes all the rows of a table.
- The table structure remains intact.
- It is a DDL command.
- It cannot be rolled back (undone).
- A table that is referenced by a foreign key cannot be truncated unless the foreign key constraint is disabled first.

```
TRUNCATE TABLE octcustomer;

TRUNCATE TABLE table_name;     -- all rows, cannot be restored
DELETE FROM table_name;        -- all rows, can be restored (rolled back)
```

3.4 DROP

The DROP command deletes the whole table (data and structure).

- A normal DROP removes the table, but the table goes to the recycle bin and can be restored from there.

- Using PURGE deletes the table permanently. It is not found in the recycle bin.

```
DROP TABLE octcustomer;          -- deleted, but can be restored from the recycle bin
DROP TABLE octcustomer PURGE;    -- deleted permanently (like Shift + Delete)
```

4. DML Commands: INSERT, UPDATE and DELETE

DML (Data Manipulation Language) commands add, change and remove the **data** inside a table.

4.1 INSERT Command

The INSERT command is used to add new rows to a table. A course table is used for the examples.

```
CREATE TABLE course (
 course_id      INT PRIMARY KEY,
 course_name    VARCHAR2(50) NOT NULL,
 fees           INT,
 course_duration INT,
 enquiry_date   DATE
);
```

Different ways to insert data

```
-- 1. Using only values (in the same order as the columns)
INSERT INTO course VALUES (1, 'JavaFSD', 50000, 6, '13-oct-26');
SELECT * FROM course;

-- 2. Using the optional column list (any order)
INSERT INTO course (enquiry_date, fees, course_duration, course_id, course_name)
VALUES ('12-oct-26', 55000, 6, 2, 'PythonFSD');

-- 3. Using the TO_DATE function
INSERT INTO course (enquiry_date, fees, course_duration, course_id, course_name)
VALUES (TO_DATE('12-oct-26', 'dd-mon-yy'), 53000, 3, 3, 'DotNet');

-- 4. Using the SYSDATE function (current date)
INSERT INTO course (enquiry_date, fees, course_duration, course_id, course_name)
VALUES (SYSDATE, 53000, 5, 4, 'MERN');

-- 5. NULL is inserted automatically when a column is left out
INSERT INTO course (enquiry_date, fees, course_id, course_name)
VALUES (SYSDATE, 57000, 5, 'Testing');

-- 6. Inserting NULL manually
INSERT INTO course (enquiry_date, fees, course_duration, course_id, course_name)
```

```
VALUES (SYSDATE, 53000, NULL, 6, 'DSA');
```

Using substitution variables

The values are entered by the user at run time. Character and date values must be written in single quotes, for example `'&course_name'` and `'&enq_date'`.

```
INSERT INTO course
VALUES (&course_id, '&course_name', &course_fees, &course_dur, '&enq_date');
```

Inserting data using a subquery

Rows can be copied from another table with `INSERT ... SELECT`. The number of columns and their data types must match.

```
INSERT INTO course (course_id, course_name, fees, course_duration, enquiry_date)
SELECT employee_id, email, salary, manager_id, hire_date
FROM employees
WHERE last_name = 'Ernst';
```

Data in the `course` table after the inserts:

course_id	course_name	fees	course_duration	enquiry_date
1	JavaFSD	50000	6	13-10-26
2	PythonFSD	55000	6	12-10-26
3	DotNet	53000	3	12-10-26
4	MERN	53000	5	13-10-26
5	Testing	57000	NULL	13-10-26
6	DSA	53000	NULL	13-10-26
7	JavaCore	20000	3	15-10-26
8	data analytics	5000	6	12-10-26
104	BERNST	6000	103	21-05-91

4.2 UPDATE Command

The `UPDATE` command is used to **modify existing data** in a table.

> **In simple words:** UPDATE changes values that are already stored. Always use a WHERE clause to choose which rows to change.
>
> **Important:** If the `WHERE` clause is not used, **all rows** in the table are modified with the given value.

```
SELECT * FROM course;

-- Applies only to the rows that satisfy the WHERE condition
```

```
UPDATE course SET course_duration = 6 WHERE course_id = 5;
UPDATE course SET course_duration = 6 WHERE course_duration IS NULL;
UPDATE course SET course_duration = 12 WHERE course_name = 'PythonFSD';

-- Without WHERE: applies to all rows
UPDATE course SET course_duration = 5;
```

UPDATE using a subquery

```
UPDATE course
SET course_duration = 12
WHERE course_id = (SELECT course_id FROM course WHERE course_name = 'JavaFSD');

UPDATE course
SET course_duration = (SELECT manager_id FROM employees WHERE first_name =
'Neena')
WHERE course_id = (SELECT course_id FROM fsdcourse WHERE course_name = 'DA');
```

4.3 DELETE Command

The `DELETE` command removes rows from a table.

> **In simple words:** DELETE removes rows, not the table. With a WHERE clause it removes only the chosen rows.

```
DELETE FROM course;                    -- deletes all records
DELETE FROM course WHERE course_id = 1;    -- deletes one particular row

DELETE FROM course
WHERE course_id = (SELECT course_id FROM course WHERE course_name = 'JavaFSD');
```

4.4 DELETE compared with TRUNCATE

Feature	DELETE	TRUNCATE
Category	DML	DDL
Rows removed	All rows, or only rows chosen with WHERE	All rows only
Table structure	Remains	Remains
Can it be undone?	Yes, it can be rolled back	No, it cannot be rolled back

5. Database Transactions

5.1 What is a Transaction?

A group of operations that is treated as **one logical unit** is called a **transaction**.

> **In simple words:** A transaction is an all-or-nothing piece of work. Either all its changes are saved, or all are undone.

A database transaction consists of ONE of the following

- DML statements that make up one consistent change to the data.
- One DDL statement.
- One DCL statement.

When does a transaction begin?

A transaction begins when the **first DML statement** is executed.

When does a transaction end?

- A COMMIT or ROLLBACK statement is issued.
- A DDL or DCL statement executes (automatic commit).
- The user exits SQL Developer or SQL*Plus.
- The system crashes.

5.2 Transaction Operations (TCL)

Command	Purpose
COMMIT	Saves the data permanently (100%).
ROLLBACK	Undoes the changes made in the database.
SAVEPOINT	Creates a marker for a partial operation.

5.3 COMMIT and ROLLBACK

```
CREATE TABLE empvalues
(
 emp_id   INT PRIMARY KEY,
 emp_name VARCHAR2(50),
 emp_des  VARCHAR2(30),
 salary   INT
);

INSERT INTO empvalues VALUES (100, 'Anuradha', 'Software Trainer', 25000);
INSERT INTO empvalues VALUES (101, 'Ganesh', 'Systech', 20000);
INSERT INTO empvalues VALUES (102, 'Milind', 'Software Trainer', 25000);
INSERT INTO empvalues VALUES (103, 'Sagar', 'Software Trainer', 25000);
INSERT INTO empvalues VALUES (104, 'Saniya', 'Software Trainer', 25000);

SELECT * FROM empvalues;
COMMIT;
```

```
DELETE FROM empvalues WHERE emp_id = 104;
UPDATE empvalues SET emp_des = 'Testing Trainer' WHERE emp_id = 103;
ROLLBACK;      -- both changes are undone
```

- ROLLBACK restores the data to the point of the last COMMIT.
- ROLLBACK is possible only until a COMMIT is done.
- If a record is deleted or updated and then committed, ROLLBACK does not work. The change is permanently saved.

5.4 SAVEPOINT

A SAVEPOINT creates a temporary point inside a transaction. It allows you to roll back **part** of a transaction instead of undoing all the changes.

> **In simple words:** A savepoint is like a checkpoint in a game. If something goes wrong, you can return to that checkpoint instead of starting again.

Syntax:

```
SAVEPOINT savepoint_name;
ROLLBACK TO savepoint_name;
```

Command	Purpose
SAVEPOINT	Creates a checkpoint inside a transaction.
ROLLBACK TO	Undoes the changes made after that checkpoint.
COMMIT	Permanently saves the transaction's changes.

```
UPDATE empvalues SET salary = 50000 WHERE emp_id = 101;

SAVEPOINT sp1;
UPDATE empvalues SET salary = 50000 WHERE emp_id = 102;

ROLLBACK TO sp1;   -- undoes only the update of employee 102
ROLLBACK;          -- undoes everything since the last COMMIT

SAVEPOINT sp2;
DELETE FROM empvalues WHERE emp_id = 100;
ROLLBACK TO sp2;   -- the deleted row is restored
```

What happens when you roll back to an earlier savepoint?

When you roll back to an earlier savepoint, **all savepoints created after it are erased**. Consider this sequence:

```
SAVEPOINT sp1;

UPDATE empvalues SET salary = 60000 WHERE emp_id = 101;
```

```
SAVEPOINT sp2;

DELETE FROM empvalues WHERE emp_id = 100;

ROLLBACK TO sp1;
```

After `ROLLBACK TO sp1`:

- The salary update to 60000 is undone.
- The deletion of employee 100 is undone.
- `sp2` is erased, because it was created after `sp1`.
- `sp1` remains available.

> **Note:** You cannot run `ROLLBACK TO sp2` afterwards in this transaction, because `sp2` no longer exists.

5.5 Quick Recap of Command Categories

Category	Use
DDL	Creates objects (create, alter, drop).
DML	Adds and changes data (insert, update, delete).
TCL	Saves or undoes the data (commit, rollback, savepoint).
DCL	Gives permission to access the data (grant, revoke).

6. Controlling User Access (DCL)

DCL (Data Control Language) commands control who can connect to the database and what they are allowed to do.

> **In simple words:** A user is a database login. A privilege is a permission. GRANT gives a permission to a user.

6.1 Creating a User and Granting Privileges

```
CREATE USER seeduser
IDENTIFIED BY your_password;

GRANT CONNECT TO seeduser;
GRANT RESOURCE TO seeduser;
GRANT CREATE TABLE TO seeduser;
```

After connecting as the new user, test what the user can do:

```
SELECT * FROM employees;     -- may fail, because the user has no access to that table
```

```
CREATE TABLE stud (
  roll INT,
  name VARCHAR2(20)
);

SELECT * FROM stud;
INSERT INTO stud VALUES (1, 'ABC');
```

Commands used

Command	Purpose
CREATE USER	Creates a database user.
IDENTIFIED BY	Specifies the user's password.
GRANT	Assigns privileges or roles.
CREATE ROLE	Creates a new role.

> **Note:** Commands such as CREATE USER and CREATE ROLE require suitable administrative privileges.

6.2 Roles

A **role** is a named group of privileges that can be granted to one or more users.

> **In simple words:** Instead of granting the same permissions to every user one by one, put them in a role and give the role to the users.

- A role can be granted to multiple users.

```
CREATE ROLE staff;
GRANT CONNECT, RESOURCE, CREATE TABLE TO staff;

CREATE USER officestaff
IDENTIFIED BY your_password;

GRANT staff TO officestaff;
```

6.3 Changing a Password and Dropping a User

```
-- Change the password of a user
ALTER USER officestaff
IDENTIFIED BY new_password;

-- Drop a user (the user must be disconnected first)
DROP USER officestaff;
```

7. Views

7.1 What is a View?

A **view** logically represents a subset of data from one or more tables.

> **In simple words:** A view is like a shadow or a window of a table. It does not store the data itself; it shows the data of the table underneath.

Advantages of views

- To restrict data access.
- To make complex queries easy.
- To provide data independence.
- To present different views of the same data.

7.2 Types of Views

- **Simple view:** data comes from a single table.
- **Complex view:** data comes from one or more tables, and it can use functions and groups of data.

Feature	Simple views	Complex views
Number of tables	One	One or more
Contain functions	No	Yes
Contain groups of data	No	Yes
DML operations through a view	Yes	Not always

7.3 Creating Views

```
-- Simple view
CREATE VIEW devcop
AS
SELECT * FROM developer;

-- DML through a simple view works
INSERT INTO devcop VALUES (103, 'Ganesh', 'graduate', 'g@gmail.com', 33);

-- Show all the views you have created
SELECT * FROM user_views;

-- Complex view (it uses a group function)
CREATE VIEW totaldev
```

```
AS
SELECT COUNT(*) AS totalemp FROM developer;
```

DML on a complex view

The totaldev view has only one column, so inserting a row with five values fails. DML is also not allowed on a view that is built with a group function.

```
INSERT INTO totaldev VALUES (103, 'Ganesh', 'graduate', 'g@gmail.com', 33);

-- Error report:
-- SQL Error: ORA-00913: too many values
```